

PROJECT REPORT

Task Management System

A Role-Based Web Application for Employee Task Assignment and Tracking

Submitted by

Sushant Tiwari

Domain: Web Application Development (Flask, MySQL)

Focus Area: Full-Stack Development, AI/ML & Data Analytics

July 2026

1. Objective

The goal of this project is to build a small, internal Task Management System that lets authorized personnel (admin and manager roles only) log in, assign one of a fixed set of task titles to an employee, and track completion status through a simple dashboard. There is no public sign-up flow — access is restricted to seeded accounts — and the system is intentionally scoped for internal team use rather than public deployment.

Specifically, the system is designed to:

- Authenticate users against a MySQL-backed login table, restricted to the admin and manager roles.
- Allow an authenticated user to assign a task (from a fixed list of titles) to any employee on record.
- Track each task's completion status and surface a live count of total / completed / pending tasks.
- Keep the codebase deployable — environment-driven configuration, no secrets committed to source, and no crash paths on invalid input.

2. Tech Stack / Libraries Used

As this is a web application rather than a machine-learning project, there is no dataset or trained model involved. The table below lists the technologies and libraries used in place of an ML library list.

Component	Technology	Purpose
Backend framework	Flask 3.0.3	Routing, templating, session management
Database	MySQL	Persistent storage for login, employee, and task records
DB driver	mysql-connector-python 9.0.0	Python ↔ MySQL connectivity
Config management	python-dotenv 1.0.1	Loads .env values into environment variables
Security	Werkzeug 3.0.3	Password hashing (generate_password_hash / check_password_hash)
Testing	pytest 8.3.2	Automated route- and logic-level test suite
Frontend	HTML5, CSS3, vanilla JavaScript	Login form, task dashboard, client-side validation

3. Database Design (in place of Dataset)

The system uses three related MySQL tables, defined in database.sql:

Table	Key Columns	Notes
login	id (PK), username (unique), password (hashed), role (admin/manager)	role is a MySQL ENUM restricted to admin/manager
employee	id (PK), employee_name, email (unique), department	Populated with 4 seed employees across 4 departments
task	id (PK), employee_id (FK → employee.id), title, completed, created_at	ON DELETE CASCADE: removing an employee removes their tasks

Seed data ships with two demo accounts (admin/admin123 and manager/manager123), stored as pbkdf2:sha256 password hashes rather than plaintext.

4. Methodology

The application follows a straightforward request/response flow with server-side validation at every step:

- Login ("/", GET/POST): renders the login form; on POST, looks up the user by username, verifies the submitted password against the stored hash, and confirms the role is admin or manager before creating a session.
- Session gate: every request to /tasks checks is_logged_in() and redirects unauthenticated users back to the login page.
- Task dashboard ("/tasks", GET): loads the employee list and the task list (joined with employee data) from MySQL, and computes total / completed / pending counts for the metric cards.
- Task assignment ("/tasks", POST): validates employee_id (must be numeric), task_title (must be non-empty), and completed (must be "true"/"false"); on success, inserts the row and redirects back to /tasks using the Post/Redirect/Get pattern so a page refresh cannot create a duplicate task.
- Logout ("/logout"): clears the Flask session and redirects to login.

5. System Architecture (in place of Model Architecture)

There is no ML model in this project, so this section covers the application's architecture instead.

Layer	Responsibility
Presentation	templates/login.html, templates/tasks.html + static/css/style.css, static/js/script.js — form rendering and lightweight client-side validation

Layer	Responsibility
Application	app.py / main.py — Flask routes, session handling, input validation, password hashing
Data	MySQL database task_management_system — login, employee, task tables
Configuration	.env (loaded via python-dotenv) — DB credentials, FLASK_SECRET_KEY, FLASK_DEBUG

6. Testing & Results

An automated test suite (tests/test_samples.py) covers the application logic using pytest and mocked database connections, so tests run without requiring a live MySQL instance:

Test Area	Cases Covered	Result
Login	Empty fields, correct hashed password, wrong password, disallowed role, DB error handling	5/5 passed
Task management	Redirect when logged out, page load, non-numeric employee id, missing fields, successful insert redirect	5/5 passed
Logout	Session cleared on logout	1/1 passed

Total: 12/12 automated tests passed. The application was also manually verified end-to-end against a local MySQL instance seeded from database.sql: login with both demo accounts, task assignment, dashboard counts updating correctly, and session clearing on logout.

7. Bugs Identified & Fixed

A code review of the original implementation surfaced the following issues, all of which were corrected:

- Plaintext password storage and comparison — replaced with Werkzeug password hashing.
- Hardcoded fallback secret key committed to source — replaced with a randomly generated per-process key when FLASK_SECRET_KEY is unset.
- README and code disagreed on debug mode (debug=True/use_reloader=True in code vs. debug=False claimed in docs) — unified behind a single FLASK_DEBUG environment variable, defaulting to False.
- Unhandled ValueError when a non-numeric employee id was submitted, causing an HTTP 500 — fixed with explicit input validation.
- Raw database exceptions rendered directly to the browser — now logged server-side; the user sees a generic, safe message.
- Successful task submissions re-rendered the form instead of redirecting, allowing duplicate task creation on page refresh — fixed using the Post/Redirect/Get pattern.

- templates/, static/, and test files were referenced by the app but were missing from the project — built out and verified against the running application.

8. Conclusion

The Task Management System successfully meets its objective: a small, role-restricted web application for assigning and tracking employee tasks, backed by a normalized MySQL schema with a proper foreign-key relationship. The original implementation's core design — parameterized SQL queries, role-gated session authentication, and a clean three-table schema — was sound and free of SQL-injection risk. The issues found were primarily around security hygiene (plaintext passwords, a hardcoded secret key) and robustness (an unhandled crash path, a documentation/code mismatch, and a missing frontend layer), all of which have been addressed. The system is now covered by a 12-test automated suite and has been verified to run end-to-end.

As a possible extension, the application could add CSRF protection on forms and rate-limiting on the login endpoint before being exposed beyond a trusted internal network.